

## Week 2: Nonparametric Learning

CSCI 447: Machine Learning– Fall 2026

1

### Classification

- Suppose we are presented with a query point  $\mathbf{x}_q$  and want to know the class to which it belongs.
- A probabilistic approach:

$$c = \underset{c \in \mathcal{C}}{\operatorname{argmax}} P(c|\mathbf{x}_q).$$

where

$$P(c|\mathbf{x}_q) = \frac{P(\mathbf{x}_q|c)P(c)}{P(\mathbf{x}_q)} = \frac{P(\mathbf{x}_q|c)P(c)}{\sum_{c' \in \mathcal{C}} P(\mathbf{x}_q|c')P(c')}$$

- Thus we need to estimate  $P(\mathbf{x}_q|c)$  and  $P(c)$ .

2

## Parzen Window Classification

- Given  $\mathcal{X} = \langle \mathbf{x}_i, c_i \rangle_{i=1}^n = \langle \{x_i^1, \dots, x_i^d\}, c_i \rangle_{i=1}^n$
- Find the class to which query point  $\mathbf{x}_q$  belongs.

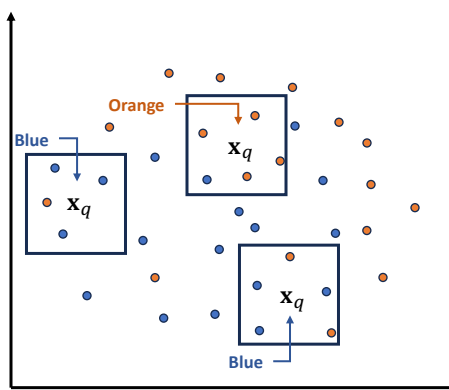
$$H(\mathbf{u}) = \begin{cases} 1 & |u_j| < \frac{1}{2}, j = 1, \dots, d \\ 0 & \text{otherwise} \end{cases}$$

where

$$\mathbf{u} = \frac{\mathbf{x} - \mathbf{x}_q}{h^d}, \quad K_c(\mathbf{x}_q) = \sum_{t=1}^n H\left(\frac{\mathbf{x}_q - \mathbf{x}^t}{n}\right) \Big|_c, \quad c = \operatorname{argmax}_c K_c(\mathbf{x}_q).$$

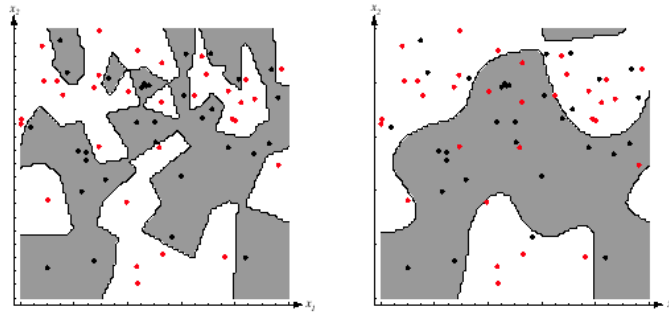
3

## Parzen Window



4

## Example Parzen Window



(Duda, Hart, & Stork, 2001)



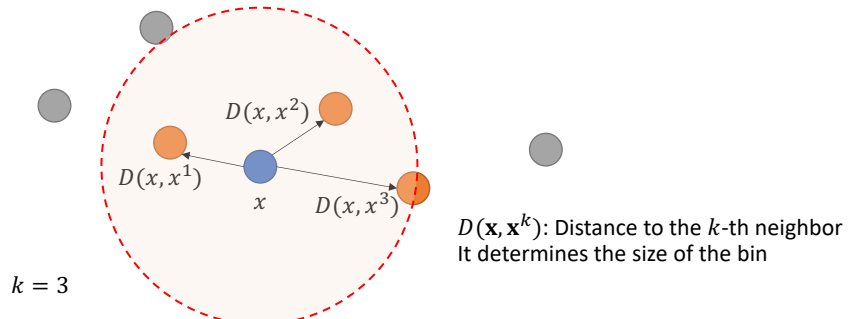
MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

5

## $k$ -Nearest Neighbor Estimation

- Doesn't need to fix the size of the bin
- The parameter  $k$  specifies how many of the neighbors are used in the estimate
- $D(\mathbf{x}, \mathbf{x}^t)$ : Distance function between  $\mathbf{x}$  and  $\mathbf{x}^t$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

6

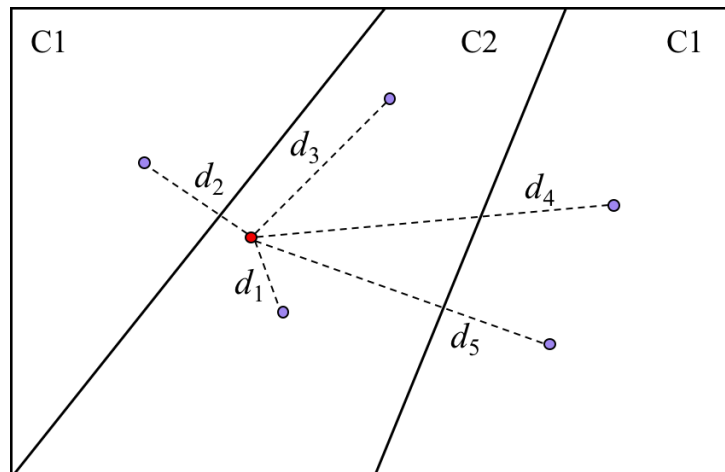
## Nearest Neighbor Classification

- Assume that  $k = 1$
- Given  $\mathcal{X} = \langle \{x_i^1, \dots, x_i^d\}, c_i \rangle_{i=1}^n$
- Find the class to which query point  $\mathbf{x}_q$  belongs.
- The nearest neighbor rule states,

Find  $\mathbf{r} \in \mathcal{X}$  such that  $\forall \mathbf{x} \in \mathcal{X}, \mathbf{x} \neq \mathbf{r}, D(\mathbf{x}_q, \mathbf{r}) < D(\mathbf{x}_q, \mathbf{x})$ , and return the class label  $c \in \mathcal{C}$  associated with example  $\mathbf{r}$ .

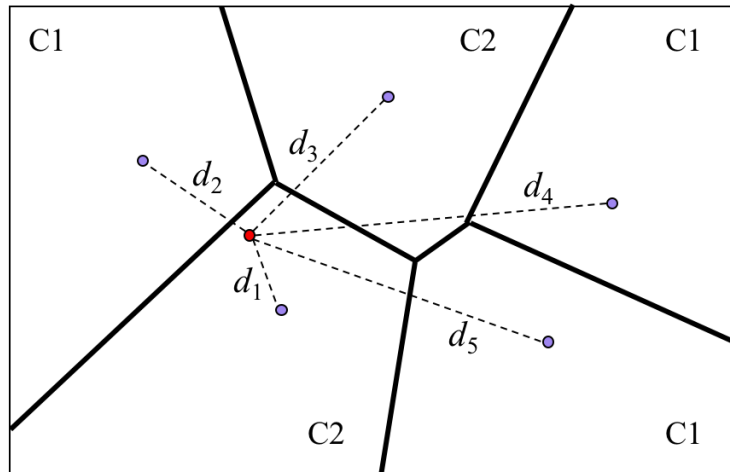
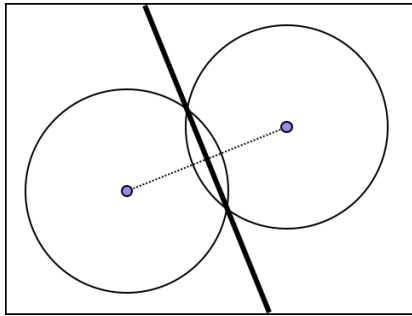
7

## Example



8

## Voronoi Diagrams



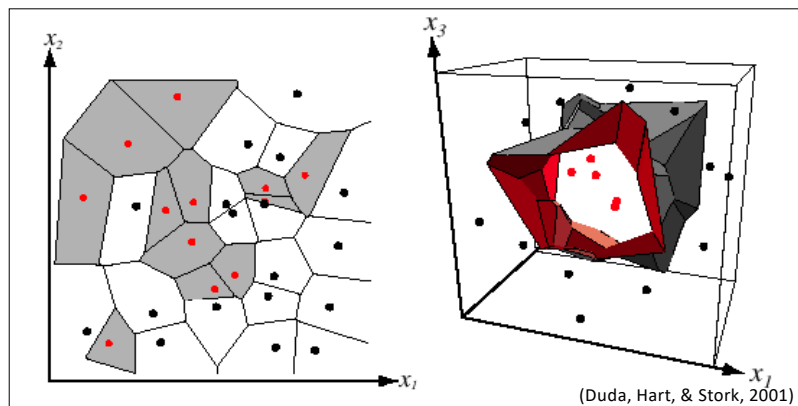
MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

9

## Nearest Neighbor Classifier

Voronoi diagram



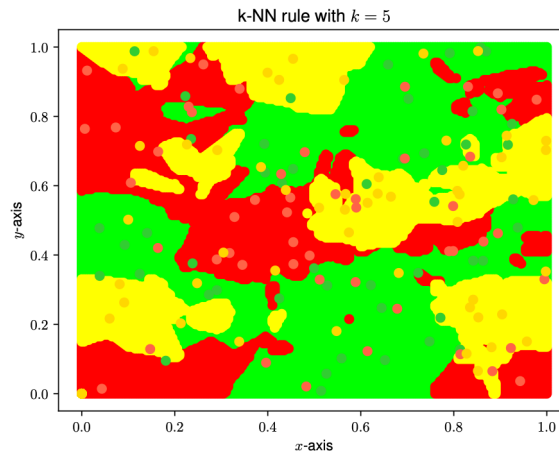
MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

10

# Nearest Neighbor Classifier

Voronoi diagram



(Sayed, 2022)



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

11

## Distance Metrics

- Metric Spaces

1. Non-negative:  $D(\mathbf{x}, \mathbf{y}) \geq 0$
2. Reflexive:  $D(\mathbf{x}, \mathbf{y}) = 0$  if and only if  $\mathbf{x} = \mathbf{y}$
3. Symmetric:  $D(\mathbf{x}, \mathbf{y}) = D(\mathbf{y}, \mathbf{x})$
4. Triangle Inequality:  $D(\mathbf{x}, \mathbf{y}) + D(\mathbf{y}, \mathbf{z}) \geq D(\mathbf{x}, \mathbf{z})$

- Minkowski Metrics:  $D_p(\mathbf{x}, \mathbf{y}) = (\sum_{i=1}^d |x_i - y_i|^p)^{1/p}$

- Manhattan Distance:  $D_1(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i|$
- Euclidean Distance:  $D_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$
- Max-coordinate Distance:  $D_\infty(\mathbf{x}, \mathbf{y}) = \max_i |x_i - y_i|$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

12

## The Value Difference Metric (Classification)

- Let  $f(\mathbf{x})$  be a feature of data instance  $\mathbf{x}$ . We assume  $f(\mathbf{x}) \in \{v_1, \dots, v_k\}$
- Let  $C_i = \#\{f(\mathbf{x}) = v_i\}$  be the count of the number of times in data set  $\mathcal{D}$  that  $f(\mathbf{x})$  takes on the value of  $v_i$ .
- Let  $C_{i,a} = \#\{f(\mathbf{x}) = v_i \wedge \text{class} = a\}$  be the count of the number of times in data set  $\mathcal{D}$  that  $\mathbf{x}$  is a member of class  $a$  and  $f(\mathbf{x})$  takes on the value of  $v_i$ .
- For feature  $f$ , define the distance between feature values  $v_i$  and  $v_j$  as follows:

$$\delta(v_i, v_j) = \sum_{a=1}^{\text{num classes}} \left| \frac{C_{i,a}}{C_i} - \frac{C_{j,a}}{C_j} \right|^p$$

- Then the distance between two data instances  $\mathbf{x}$  and  $\mathbf{y}$  is defined to be

$$D_{cat}(\mathbf{x}, \mathbf{y}) = \left( \sum_{k=1}^d \delta(\mathbf{x}_k, \mathbf{y}_k) \right)^{1/p}$$

## Mixed Data Types

- Suppose we have data instances  $\mathbf{x}, \mathbf{y}$  consisting of a combination of categorical and numerical features.
- Remember to normalize the numerical features first (to range 0 ... 1).
- Let  $D_{num}(\mathbf{x}, \mathbf{y})$  be the distance between the numerical part of the two instances.
- Let  $D_{cat}(\mathbf{x}, \mathbf{y})$  be the distance between the categorical part of the two instances.
- Calculate a combined distance as follows:

$$D(\mathbf{x}, \mathbf{y}) = \sqrt[p]{(D_{num}(\mathbf{x}, \mathbf{y}))^p + (D_{cat}(\mathbf{x}, \mathbf{y}))^p}$$

- Since the features are “normalized,” this effectively combines the base components of the distance calculations. (Note the  $p$ -th root isn't necessary!)

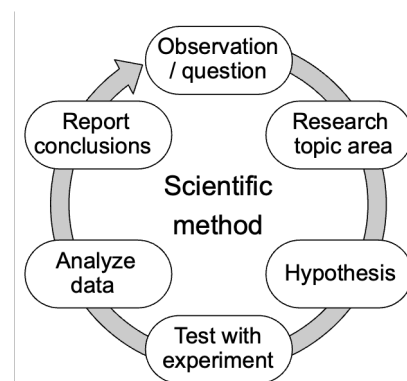
## Experimental Methods

- The Hypothesis
- A Testable Measure
- Experiments
  - Hold-Out
  - Cross-Validation
- Tuning
  - Factorial Design
  - Self-Adaptive Systems

15

## Scientific Method and ML

- Science is observational (inductive)
- Machine learning is also observational (inductive)
- Science is hypothesis-driven
- Machine learning is also hypothesis-driven
- In machine learning, “models” are often referred to as “hypotheses”



[https://commons.wikimedia.org/wiki/File:The\\_Scientific\\_Method.svg](https://commons.wikimedia.org/wiki/File:The_Scientific_Method.svg)

16



# The Hypothesis

Key to well-constructed research is defining a strong, testable hypothesis.

- Defines the scope of the research
- Provides means to keep research objective
- Determines the means to evaluate results
- Enables quantifiable (statistical) methods to be employed
- Facilitates understanding
- Facilitates determining significance/usefulness

17

# The Hypothesis

- Narrow scope:
  - Poor: Image classification.
  - Better: Classification of cloud types based on evolutionary spectral feature selection in deep learning.
- Quantifiable:
  - Poor: Algorithm *X* will be better than algorithm *Y*.
  - Better: Algorithm *X* when augmented with local search will yield a significant increase in both precision and recall than without the augmentation.

18

## The Hypothesis

- Understandable:
  - Poor: Reliable extraction of redaction hermeneutics in the *Textus Receptus*.
  - Better: Accurate identification of editorial preferences in ancient Greek texts, such as the *Textus Receptus*.
- Significant and Useful:
  - Poor: Accurate classification of fruit fly geometry.
  - Better: Improved accuracy in gene alignment based on frequented regions in De Bruijn Graphs.

## A Testable Measure

- Useful measures in machine learning include:
  - Accuracy
  - Precision, recall, F1
  - Cross-entropy
  - Area under the ROC curve (AUC)
  - Mean squared error
  - Mean absolute error
- All are quantifiable and testable.
- Now have the basis to design an experiment.

## Experiments

- Single run:
  - Define parameters for algorithm(s) and run on target problem(s).
  - Compare results.
- How were the parameters selected?
- Sufficient to draw conclusions?

## Experiments

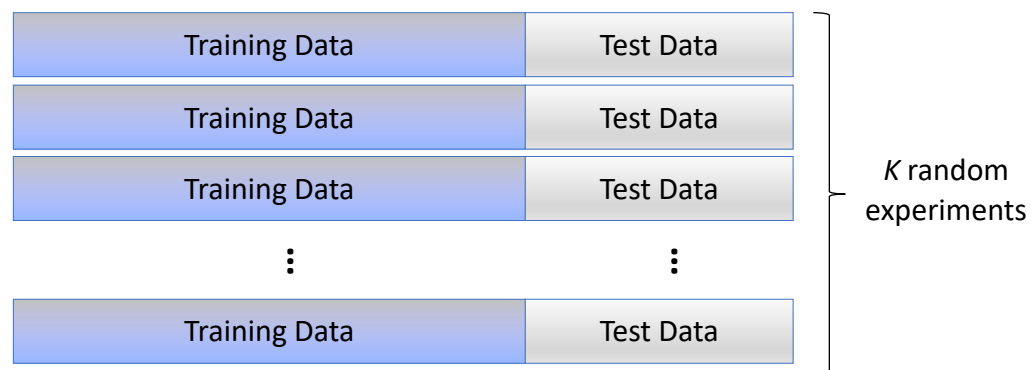
- Multiple runs:
  - Machine learning algorithms generally seek to optimize a loss function.
  - Run several independent trials to generate a sample for the target evaluation measure(s).
  - Compare distributions of the results.
- Does this address parameter selection?
- Does this address the ability to draw conclusions?

## Experiments

- Cross Validation:
  - Applies to “supervised learning” or “model building” types of problems
  - Divide data into “training set” and “test set”
  - Evaluation should focus on the test set – it tests generalization
- Does this address parameter selection?
- Does this address ability to draw conclusions?
- How does this compare to multiple runs?

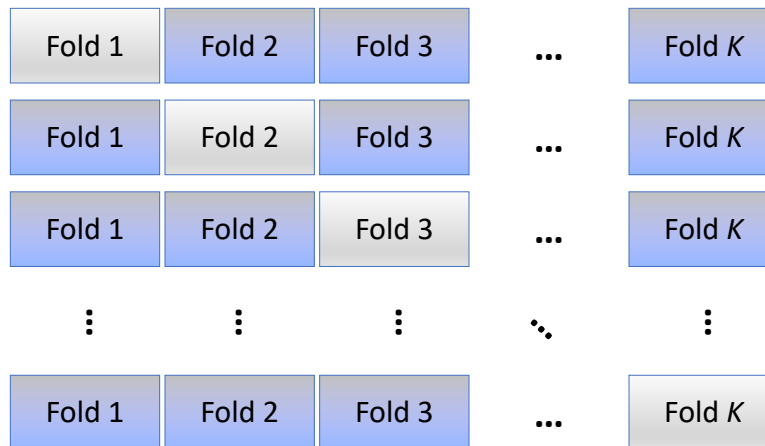
23

## Experiments – Repeated Hold Out



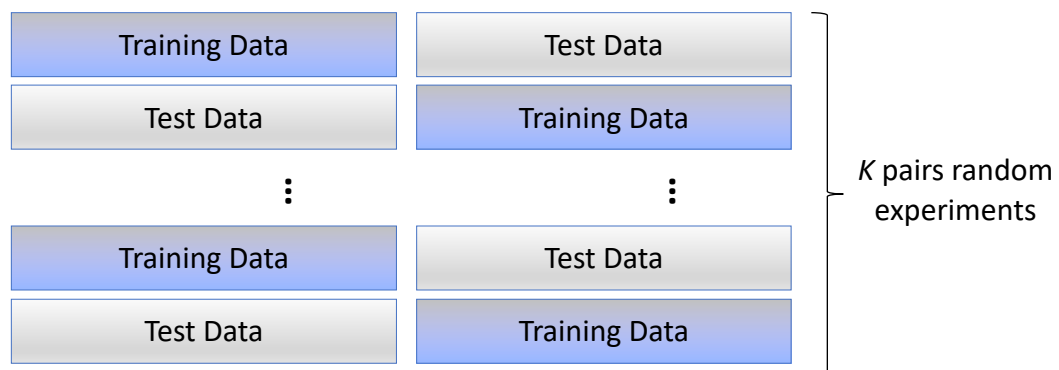
24

## Experiments – $K$ -fold Cross Validation



25

## Experiments – $K \times 2$ Cross Validation



26

## Regression [For Now]

- Null models:
  - Classification: Return the most frequent class
  - Regression: Return the average of the response values
- In the context of  $k$ -nearest neighbor
  - Classification: Do a simple majority vote of the  $k$  neighbors
  - Regression: Do a simple arithmetic mean of the  $k$  neighbors
- We will change the regression rule to provide a “weighted” mean
- The weighting function is called a “kernel”

27

## Confusion Matrices

- Suppose we have a 2-class problem:

|                 | Actual True       | Actual False      |
|-----------------|-------------------|-------------------|
| Predicted True  | # True Positives  | # False Positives |
| Predicted False | # False Negatives | # True Negatives  |

- $error = \frac{FP+FN}{TP+TN+FP+FN}$ ,  $precision = \frac{TP}{TP+FP}$ ,  $recall = \frac{TP}{TP+FN}$

- $F_\beta = (1 + \beta^2) \cdot \frac{precision \cdot recall}{\beta^2 precision + recall}$ , usually  $\beta = 1$ :  $F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$

28

## Confusion Matrices

- 3-class problem:

|                   | Actual Class 1 | Actual Class 2 | Actual Class 3 |
|-------------------|----------------|----------------|----------------|
| Predicted Class 1 | 15             | 2              | 4              |
| Predicted Class 2 | 3              | 12             | 1              |
| Predicted Class 3 | 4              | 2              | 17             |

Example:

$$TP_1 = 15$$

$$TN_1 = 32$$

$$FN_1 = 7$$

$$FP_1 = 6$$

- Micro-averaging:

$$P_{micro} = \frac{\sum_c TP_c}{\sum_c TP_c + FP_c}, R_{micro} = \frac{\sum_c TP_c}{\sum_c TP_c + FN_c}$$

- Macro-averaging:

$$P_{macro} = \frac{1}{|C|} \sum_c \frac{TP_c}{TP_c + FP_c}, R_{macro} = \frac{1}{|C|} \sum_c \frac{TP_c}{TP_c + FN_c}$$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

29

## Hyperparameter Search

- Machine learning methods are used to search a space of possible models to find a model to a problem that optimizes some loss function.
- Let  $\mathbf{M}$  denote a “model” corresponding to a solution to that problem.
- Let  $F$  denote the loss function.
- Let  $\mathbf{A}$  denote the algorithm used to optimize/search.
- Let  $\lambda$  denote the set of hyperparameters to be used by the search algorithm.
- Let  $\mathbf{X}^{tr}$  and  $\mathbf{X}^{te}$  denote training and test examples respectively.
- Then hyperparameter search (i.e., tuning) solves the following

$$\lambda^* = \operatorname{argmin}_{\lambda} F(\lambda; \mathbf{M}, \mathbf{A}, \mathbf{X}^{tr}, \mathbf{X}^{te})$$



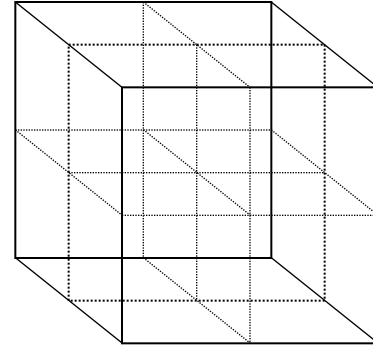
MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

30

## Grid Search (Factorial Study)

- Regard each parameter as a “factor”
- Since each parameter is real-valued, define set of discrete levels.
- Define “grid” of all combinations of levels.
  - Expensive but parallelizable
  - For  $d$  variables and  $k$  levels,  $k^d$  combinations.
  - Compromise: fix  $d - 1$  variables and vary the  $d$ th variable.



31

## Random Search

- Replaces exhaustive enumeration of parameter values.
- Selects combination of values purely at random and keeps the best.
- Easy to parallelize, as in grid search.
- Often outperforms grid search, both in terms of final parameters (need not discretize) and computational performance.

32



## Random Search (Hill Climb)

```

Initialize  $\lambda \in \mathbb{R}^{|\lambda|}$  to a random vector of hyperparameters.
While not terminate() do
    Sample a new  $\lambda' \in \mathbb{R}^{|\lambda|}$  from a hypersphere surrounding  $\lambda$ .
    If  $f(\lambda') < f(\lambda)$  then
        Set  $\lambda = \lambda'$ 
  
```

To select from hypersphere:

```

Let  $d = |\lambda|$ 
Generate  $(d+1)$ -dimensional vector from  $\mathcal{N}(0,1)^{d+1}$  denoted  $\mathbf{x}$ .
Calculate radius of  $\mathbf{x}$  as  $r_x = \sqrt{\sum_{i=1}^{d+1} x_i^2}$ .
Divide vector by radius,  $\mathbf{x}' = \mathbf{x}/r$ .
Randomly discard one of the components of this vector.
  
```

33

## Bayesian Optimization

- Idea is to use parameter search as optimizing a black-box function.
- Define a prior probability over where to sample.
- Update the prior based on the results of sampling.
- Repeat until convergence.
- Performance often beats grid search and random search in quality and computational performance; however, optimizing the acquisition function can be expensive.

34

## Bayesian Optimization

- Start by assuming a uniform prior over the hyperparameters and generate a vector  $\lambda$ .
- Add  $\lambda$  to history  $H$ .
- Evaluate performance of your algorithm with  $\lambda$ .
- Repeat
  - Select a new vector  $\lambda'$  by maximizing acquisition function  $u$ .
  - Evaluate performance with  $\lambda'$ .
  - Add  $\lambda'$  to  $H$ .

35

## Acquisition Functions

- Acquisition function effectively updates the prior based on previous experience and assessing utility of hyperparameter choices.
- Bayesian optimization often assumes a Gaussian process.
- One acquisition function is based on “expected improvement.”

$$u_{EI}(\lambda) = (\mu(\lambda) - f(\lambda^*))\Phi(Z) + \sigma(\lambda)\phi(Z)$$

where

$$Z = \frac{\mu(\lambda) - f(\lambda^*) - \xi}{\sigma(\lambda)}$$

- Here  $\Phi$  is the normal CDF and  $\phi$  is the normal PDF,  $\lambda^*$  denotes the best set of parameters seen so far, and  $\xi$  is a parameter to balance exploration and exploitation.
- Mean  $\mu(\lambda)$  and standard deviation  $\sigma(\lambda)$  are updated iteratively as samples are generated.
- Whenever  $\sigma(\lambda) = 0$ , the above equations simply go to zero.

36

## Gradient-Based Optimization

- Sometimes, the parameter tuning follows a convex function.
- Even when it doesn't one can assume it is convex and search for a local optimum.
- This amounts to a hill-climbing search.
- Strategies:
  - Generate several modifications within a neighbor and select the best.
  - Generate several modifications using random search and select the best.
  - Apply random search with simulated annealing.

37

## Meta-Evolution

- One approach to setting hyperparameters for a machine learning algorithm, is to use an evolutionary algorithm to do the selection for us.

Generate initial population of parameter vectors  $\lambda$ .  
 While not terminate() do  
     Evaluate EA using fitness function with each of the parameter vectors  
     Apply selection, mutation, crossover (or other operators)  
     Update population

- To avoid tuning the meta-EA, it is customary to apply default, nominal settings, whether this is a good idea or not.
- Otherwise, "turtles all the way down."

38

## $k$ -Nearest Neighbor Classifier

- May need to examine all examples in  $E$  (training set), especially if  $x_q$  is not in  $E$ , thus  $O(|E|)$  comparisons
- Each comparison involves computing  $D(e, x_q) \forall e \in E$  which requires  $O(d)$  calculations for  $d$  dimensions
- Thus, worst-case performance for naïve nearest neighbor search is  $O(d|E|)$
- We need methods to address this complexity

## Data Set Reduction

- Can reduce error by reducing the size of the exemplar (training) set.
- Nearest neighbor with a reduced training set is called *edited nearest neighbor*.
- There are several methods for editing the exemplar set.
- Another alternative is to replace the exemplars with “prototype” exemplars. This is called *condensed nearest neighbor*.

## Edited Nearest Neighbor

- Nearest neighbor methods are expensive.
- They are also susceptible to noisy data and irrelevant features.
- Can draw inspiration from what we call “Helpful Teacher” methods.
  - Improves computational performance
  - Focuses on questionable regions in data
- Foreshadow: “Editing” is similar to stepwise backward elimination in feature selection.

## Edited Nearest Neighbor – v1

The first approach removes misclassified examples.

1. For each  $\mathbf{x}^t \in \mathcal{X}$  do
  - a) Classify  $\mathbf{x}^t$  using one-nearest-neighbor on  $\mathcal{X} \setminus \{\mathbf{x}^t\}$
  - b) Compare class returned to true class of  $\mathbf{x}^t$
  - c) If disagree, remove  $\mathbf{x}^t$  from  $\mathcal{X}$
2. Repeat until performance degrades on separate validation data set

## Edited Nearest Neighbor – v2

The second approach removes correct examples.

1. For each  $\mathbf{x}^t \in \mathcal{X}$  do
  - a) Classify  $\mathbf{x}^t$  using one-nearest-neighbor on  $\mathcal{X} \setminus \{\mathbf{x}^t\}$
  - b) Compare class returned to true class of  $\mathbf{x}^t$
  - c) If agree, remove  $\mathbf{x}^t$  from  $\mathcal{X}$
2. Repeat until performance degrades on separate validation data set

## Condensed Nearest Neighbor

Foreshadow: Draws inspiration from stepwise forward selection.

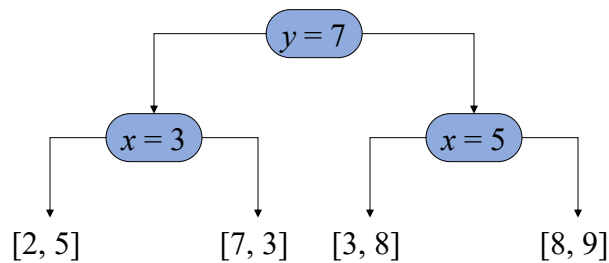
We seek to find a subset of data points  $\mathcal{Z} \subset \mathcal{X}$

1. Initialize  $\mathcal{Z} \leftarrow \text{rand}(\mathcal{X})$
2. Repeat
  - a) For each  $\mathbf{x}^t \in \mathcal{X}$  do
    - Find  $\mathbf{x}' = \underset{\mathbf{x}'' \in \mathcal{Z}}{\text{argmin}} D(\mathbf{x}'', \mathbf{x}^t)$
    - If  $\text{class}(\mathbf{x}') \neq \text{class}(\mathbf{x}^t)$ , add  $\mathbf{x}'$  to  $\mathcal{Z}$
3. Until  $\mathcal{Z}$  does not change

## KD-Trees

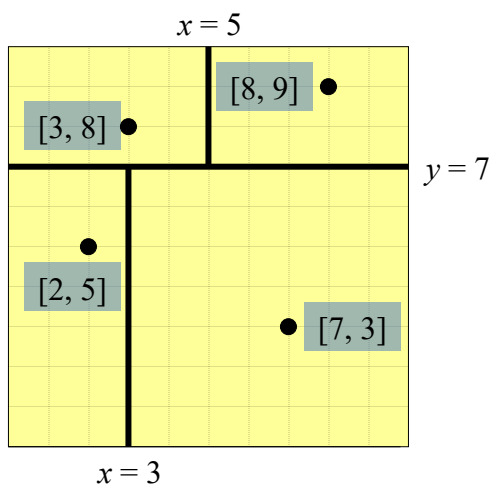
A *kd*-tree is an attempt to reduce the average complexity to  $O(d \log|E|)$

$$E = \{[7, 3], [2, 5], [3, 8], [8, 9]\}$$



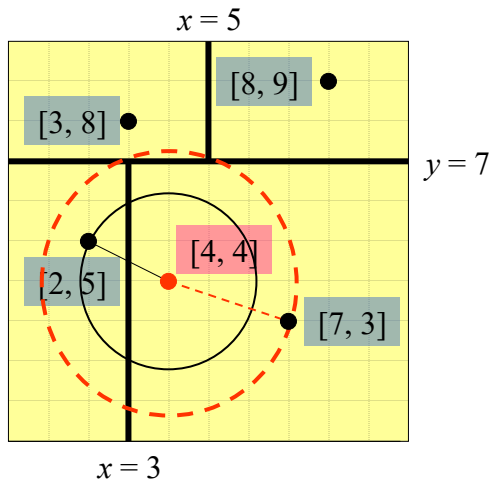
45

## KD-Trees



46

## KD-Trees



47

## The Principal Task in Nonparametric Learning

- Given a data set, we want to estimate the probability distribution for that data.
- For discrete random variables, this involves estimating the probability *mass* function.
- For continuous random variables, this involves estimating the probability *density* function.

$$P(\mathbf{X}) = \int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}'$$

48



## Recall Classification

- Suppose we are presented with a query point  $\mathbf{x}_q$  and want to know the class to which it belongs.
- A probabilistic approach:

$$c = \arg \max_{c \in \mathcal{C}} P(c|\mathbf{x}_q).$$

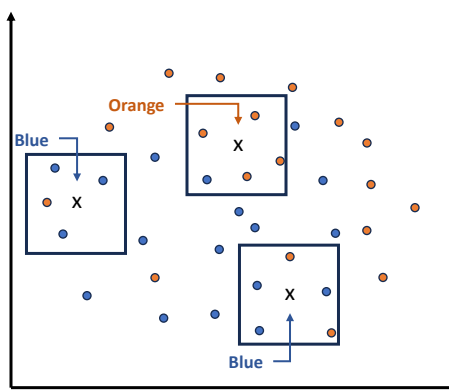
where

$$P(c|\mathbf{x}_q) = \frac{P(\mathbf{x}_q|c)P(c)}{\sum_{c' \in \mathcal{C}} P(\mathbf{x}_q|c')P(c')}$$

- Thus we have a density estimation task where we need to estimate  $P(\mathbf{x}_q|c)$  and  $P(c)$ .

49

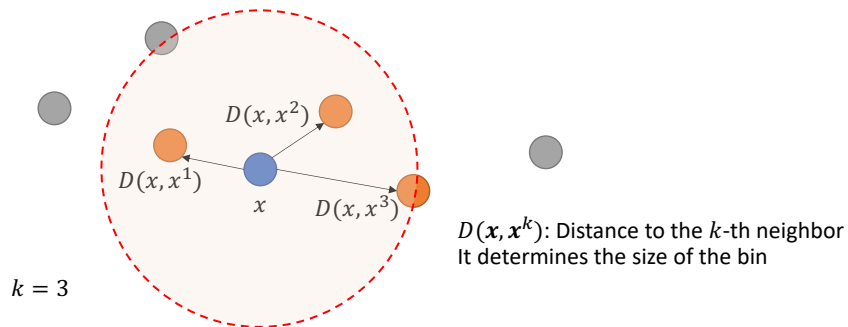
## Parzen Window Estimation



50

## $k$ -Nearest Neighbor Estimation

- Doesn't need to fix the size of the bin
- The parameter  $k$  specifies how many of the neighbors are used in the estimate
- $D(\mathbf{x}, \mathbf{x}^t)$ : Distance function between  $\mathbf{x}$  and  $\mathbf{x}^t$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

51

## Deriving an Approach

- We work off of  $n$  data samples,  $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \sim \mathbf{P}$ .
- We begin with a nonparametric assumption – we do not know a parametric form for the underlying distribution.
- Question: “What is the probability that  $k$  of the samples fall in some region  $\mathcal{R}$ .”

$$P_k(\mathbf{x}) = \binom{n}{k} P^k (1 - P)^{n-k}$$

$$E[k] = nP$$

$$\frac{k}{n} \approx P$$

- Thus, for both Parzen Windows and  $k$ -NN, the density is approximated by the points in the region defined by the window or the number of neighbors.



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

52

## Deriving an Approach

- Combining with our definition of a density function:

$$P(\mathbf{X}) = \int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}' \approx p(\mathbf{x}) V_{\mathcal{R}} \approx \frac{k}{n}$$

$$p(\mathbf{x}) \approx \frac{k/n}{V_{\mathcal{R}}} = \frac{k}{nV_{\mathcal{R}}}$$

- Assume a hypercube with side length equal to  $h$ . Then  $V_{\mathcal{R}} \approx h^d$ , so

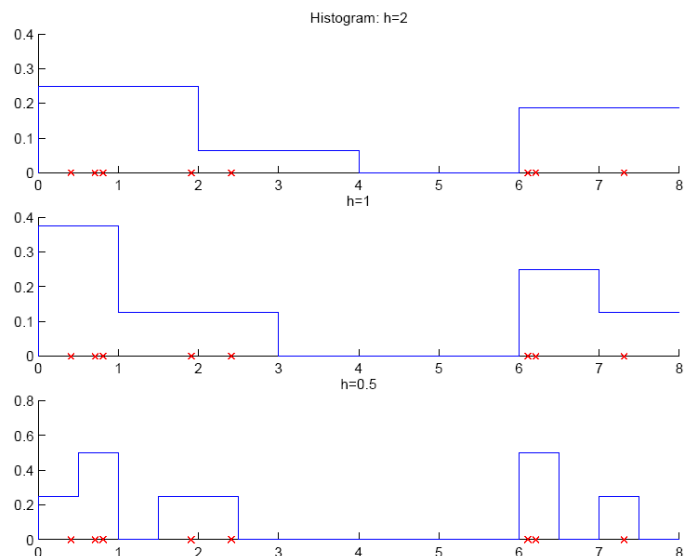
$$P(x) \approx \frac{k}{nh^d}$$

53

## Histogram Estimator

- $\mathcal{X} = \{\mathbf{x}^t\}_{t=1}^n$
- Set an origin at  $\mathbf{x}_0$ .
- Define a bin width  $h$ .
- Specify a bin as  $[\mathbf{x}_0 + mh, \mathbf{x}_0 + (m+1)h)$
- Then

$$\tilde{p}(\mathbf{x}) = \frac{1}{nh} \sum_{t=1}^n \mathbb{I}\{\mathbf{x}^t \in \text{bin}(\mathbf{x})\}$$



54

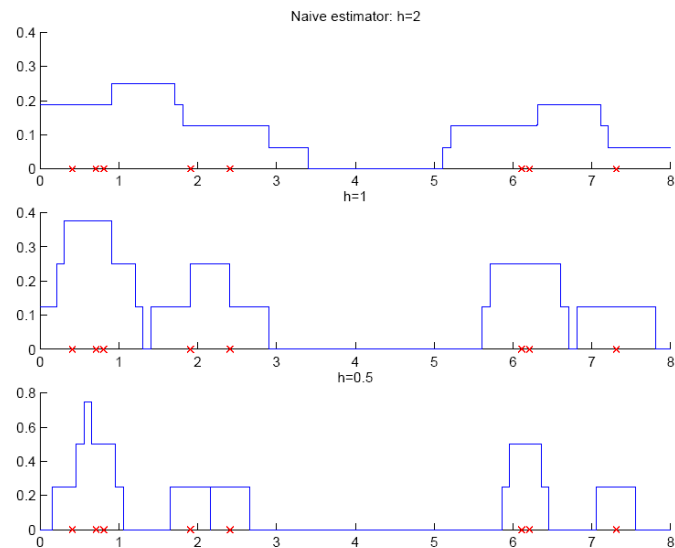
## Naïve Estimator

- $\mathcal{X} = \{\mathbf{x}^t\}_{t=1}^n$
- No origin needed
- Define a bin width  $h$ .
- Specify a bin around query point  $\mathbf{x}$  as

$$(\mathbf{x}^t - h, \mathbf{x}^t + h]$$

- Then

$$\tilde{p}(\mathbf{x}) = \frac{1}{2nh} \sum_{t=1}^n \mathbb{I}\{(\mathbf{x}^t - h, \mathbf{x}^t + h]\}$$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

55

## Weighted Estimators

We can generalize this idea.

$$\tilde{p}(\mathbf{x}) \approx \frac{1}{nh} \sum_{t=1}^n w\left(\frac{\mathbf{x} - \mathbf{x}^t}{h}\right)$$

where

$$w(\mathbf{u}) = \begin{cases} \frac{1}{2} & \text{if } |\mathbf{u}| < 1 \\ 0 & \text{otherwise} \end{cases}$$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

56

## Kernel Functions

- Weighting “function” is the basis for what we call *kernels*, denoted  $K(\mathbf{u})$ .
- The weight is determined based on similarity to a data point, where
  - $K(\mathbf{u}) \geq 0, \forall \mathbf{u}$
  - $K(-\mathbf{u}) = K(\mathbf{u}), \forall \mathbf{u}$
  - $\int_{-\infty}^{+\infty} K(\mathbf{u}) d\mathbf{u} = 1$
  - Triangle inequality need not apply
- This defines a “feature map”  $\phi : \mathcal{X} \rightarrow \mathcal{V}$  such that
 
$$K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x})^\top \phi(\mathbf{x}') \rangle_{\mathcal{V}}$$
- We will see kernels again when we talk about “support vector machines.”

57

## Kernel Estimator

This leads to what is known as “kernel density estimation” (KDE).

$$\tilde{p}(\mathbf{x}) = \frac{1}{nh} \sum_{t=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}^t}{h}\right)$$

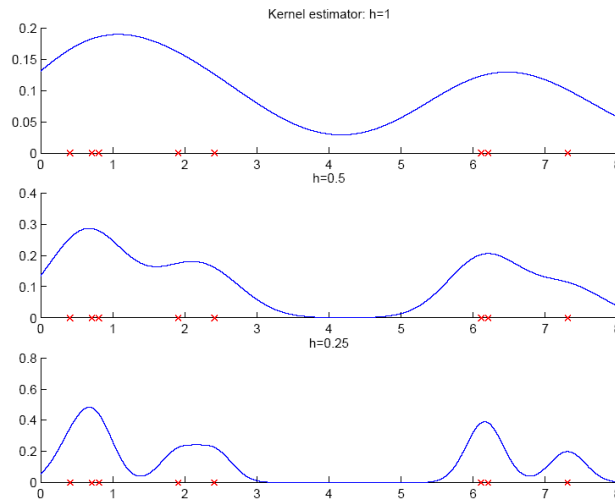
$$K(\mathbf{u}) = \frac{1}{\sqrt{2\pi}} \exp\left[-\frac{\mathbf{u}^2}{2}\right]$$

Gaussian or  
Radial Basis  
Function Kernel

$$\tilde{p}(\mathbf{x}) = \frac{1}{\sqrt{2\pi}nh} \sum_{t=1}^n \exp\left[-\frac{(\mathbf{x} - \mathbf{x}^t)^2}{2h}\right]$$

58

## Kernel Density Estimator



59

## Parzen Window Estimator

- Recall the Parzen Window. This, combined with kernels, suggests an alternative to the naïve estimator.
- Let  $\tilde{p}(\mathbf{x}) = \frac{1}{nh} \sum_{t=1}^n \mathbb{I}\{|\mathbf{x} - \mathbf{x}^t| \leq h\} K\left(\frac{\mathbf{x} - \mathbf{x}^t}{h}\right)$ .
- This reduces to  $\tilde{p}(\mathbf{x}) = \frac{1}{nh^d} \sum_{t=1}^n K\left(\frac{\|\mathbf{x} - \mathbf{x}^t\|}{h}\right) = \frac{1}{nV_{\mathcal{R}}} \sum_{t=1}^n K\left(\frac{\|\mathbf{x} - \mathbf{x}^t\|}{h}\right)$ ,  
 where  $K(\mathbf{u}) = \left(\frac{1}{\sqrt{2\pi}}\right)^d \exp\left[-\frac{\|\mathbf{u}\|^2}{2}\right]$ .
- If we include covariance, then  $K(\mathbf{u}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left[-\frac{1}{2} \mathbf{u}^T \Sigma^{-1} \mathbf{u}\right]$ .

60

## K-Nearest Neighbor Estimator

- Assume we have a distance function  $D(\mathbf{x}, \mathbf{x}')$ .
- Then

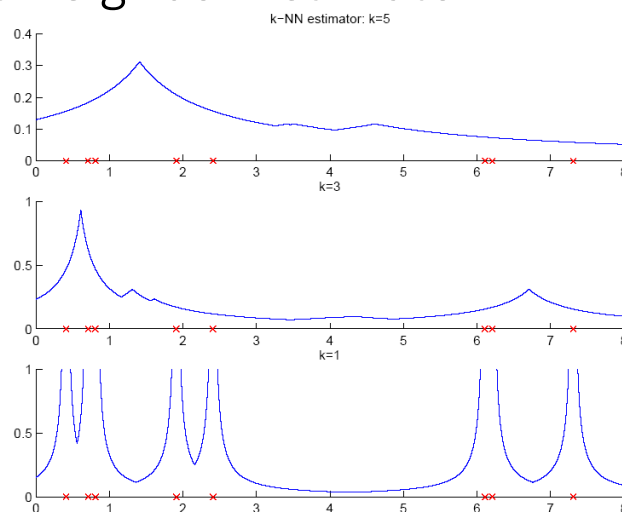
$$\tilde{p}(\mathbf{x}) = \frac{k}{2nD(\mathbf{x}, \mathbf{x}^k)}$$

$$h = D(\mathbf{x}, \mathbf{x}^k)$$

$$\tilde{p}(\mathbf{x}) = \frac{1}{nD(\mathbf{x}, \mathbf{x}^k)} \sum_{\mathbf{x}^t \in \mathcal{N}(\mathbf{x})} K\left(\frac{\|\mathbf{x} - \mathbf{x}^t\|}{D(\mathbf{x}, \mathbf{x}^k)}\right)$$

61

## K-Nearest Neighbor Estimator



62

## $k$ -Nearest Neighbor Classifier

- Using  $k$ -NN density estimation:  $\hat{p}(\mathbf{x}) = \frac{k}{2nD(\mathbf{x}, \mathbf{x}^t)}$
- Similarly, for  $\hat{p}(\mathbf{x}, c_i)$ :  $\hat{p}(\mathbf{x}, c_i) = \frac{k_i}{2nD(\mathbf{x}, \mathbf{x}^t)}$
- By the definition of conditional probability:  $\hat{p}(c_i|\mathbf{x}) = \frac{\hat{p}(\mathbf{x}, c_i)}{\hat{p}(\mathbf{x})} = \frac{k_i}{k}$
- Thus, the posterior is estimated by the fraction of samples within a  $k$ -neighborhood labeled  $c_i$
- The selected class for  $\mathbf{x}_q$  is the one with the largest count:  $c_q = \underset{c_j \in \mathcal{C}}{\operatorname{argmax}} P(c_j|\mathbf{x}_q)$

## Regression

- Suppose we are presented with a query point  $\mathbf{x}_q$  and want to compute the function value at that point.

$$\mathcal{X} = \{\mathbf{x}^t, r^t\}_{t=1}^n, \quad r^t = g(\mathbf{x}^t) + \epsilon$$

- Parametric Regression
  - Need to find coefficients in a predefined functional form
  - Minimize loss such as sum squared error
  - Leads to “least squares” regression
- Nonparametric Regression
  - Examples from data define the “shape” of the regression function
  - Compute regressed output  $\mathbf{x}_q$  at query time
  - Results in what is called a “smoother”



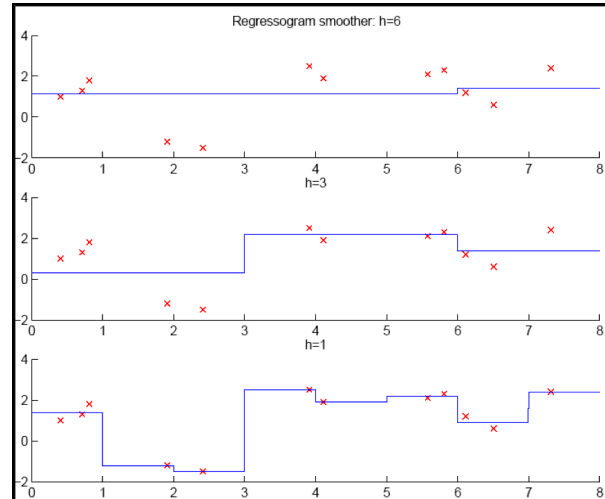
## Regressogram Smoother

- First specify an origin  $\mathbf{x}_0$  and a bin width  $h$
- Then estimate the function at a query point  $\mathbf{x}_q$  as

$$\hat{g}(\mathbf{x}_q) = \frac{\sum_{t=1}^n b(\mathbf{x}_q, \mathbf{x}^t) r^t}{\sum_{t=1}^n b(\mathbf{x}_q, \mathbf{x}^t)}$$

where

$$b(\mathbf{x}_q, \mathbf{x}^t) = \begin{cases} 1 & \text{if } \mathbf{x}^t \in \text{bin}(\mathbf{x}_q) \\ 0 & \text{otherwise} \end{cases}$$

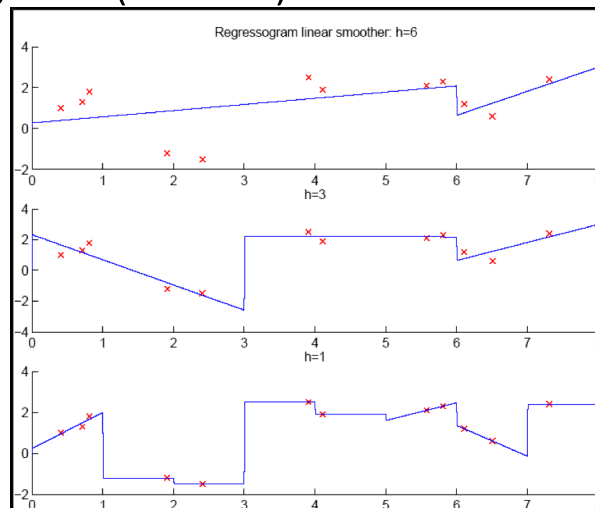


MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

65

## Regressogram (Linear) Smoother



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

- (Alpaydin, 2019)

66

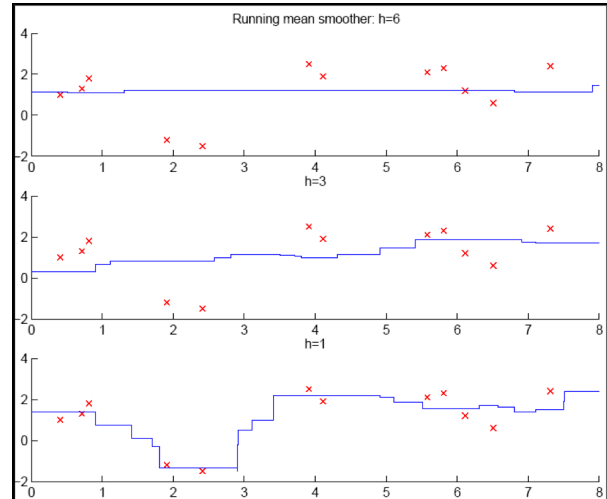
## Running Mean Smoother

Estimate the function at a query point  $\mathbf{x}_q$  as

$$\hat{g}(\mathbf{x}_q) = \frac{\sum_{t=1}^n w\left(\frac{\mathbf{x}_q - \mathbf{x}^t}{h}\right) r^t}{\sum_{t=1}^n w\left(\frac{\mathbf{x}_q - \mathbf{x}^t}{h}\right)}$$

where

$$w(\mathbf{u}) = \begin{cases} 1 & \text{if } |\mathbf{u}| < 1 \\ 0 & \text{otherwise} \end{cases}$$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

67

## Kernel Smoother

- Implements a form of locally weighted regression
- Options:
  - Select  $k$  neighbors of  $\mathbf{x}_q$  and fit a linear function
  - Select  $k$  neighbors of  $\mathbf{x}_q$  and fit a polynomial function
  - Select  $k$  neighbors of  $\mathbf{x}_q$  and fit a piece-wise linear function
- A better option is to employ a kernel function and apply a weighted average



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING

68

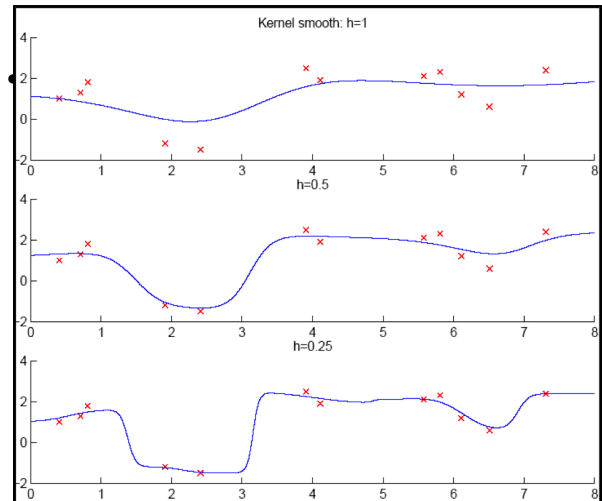
# Kernel Smoother

Estimate the function at a query point  $\mathbf{x}_q$  as

$$\hat{g}(\mathbf{x}_q) = \frac{\sum_{t=1}^n K(\mathbf{x}_q, \mathbf{x}^t) r^t}{\sum_{t=1}^n K(\mathbf{x}_q, \mathbf{x}^t)}$$

where

$$K(\mathbf{x}_q, \mathbf{x}^t) = \exp \left[ -\frac{1}{2\sigma^2} D(\mathbf{x}_q, \mathbf{x}^t) \right]$$



MONTANA  
STATE UNIVERSITY

NORM ASBJORNSON  
College of  
ENGINEERING